

Projekt Bazy Danych Zawodów Sportowych

Podstawowe założenia projektu

- **Cel projektu:**
 - Celem projektu jest stworzenie bazy danych do zarządzania informacjami związanymi z zawodami sportowymi, zawodnikami, kibicami, dyscyplinami sportowymi, wynikami, trofeami oraz wydarzeniami towarzyszącymi (np. konkursy dla kibiców, zakupy pamiątek).
 - Baza ma umożliwiać efektywne przechowywanie, wyszukiwanie i analizowanie danych związanych z zawodami sportowymi oraz ich uczestnikami.
- **Główne założenia:**
 - Baza danych ma wspierać zarządzanie informacjami o zawodnikach, kibicach, dyscyplinach sportowych, wynikach zawodów, trofeach oraz transakcjach związanych z kibicami (zakup biletów, pamiątek).
 - Projekt zakłada relacyjny model danych, który umożliwia przechowywanie powiązanych informacji w osobnych tabelach i łączenie ich za pomocą kluczy obcych.
 - Baza ma być elastyczna, aby umożliwić dodawanie nowych dyscyplin, zawodników, kibiców oraz wydarzeń sportowych.
- **Możliwości:**

- Przechowywanie danych o zawodnikach, ich dyscyplinach głównych oraz wynikach osiągniętych w zawodach.
- Zarządzanie informacjami o kibicach, ich stowarzyszeniach oraz wydatkach na bilety i pamiątki.
- Śledzenie wyników zawodów, rekordów oraz zdobytych trofeów.
- Analiza wydatków kibiców na bilety i pamiątki.
- Zarządzanie sponsorami i ich wkładem finansowym w organizację zawodów.
- **Ograniczenia:**
 - Baza danych jest zaprojektowana dla konkretnego zakresu danych (zawody sportowe, zawodnicy, kibice) i nie obejmuje innych dziedzin.
 - Niektóre tabele (np. Pamiątki, Zakupy) są ograniczone do prostych operacji i nie obsługują zaawansowanych funkcji, takich jak zarządzanie magazynem.
 - Potrzeba regularnej konserwacji – konieczne jest monitorowanie integralności danych, optymalizacja indeksów oraz tworzenie kopii zapasowych, aby zapewnić niezawodność systemu.

Schemat pielęgnacji bazy danych

1. Aktualizacja danych:

- Regularne aktualizowanie tabel Zawody, Zawodnicy, Kibice oraz Rekord100m w miarę pojawiania się nowych danych.
- Dodawanie nowych rekordów do tabel Trofea, KibicZawody, Zakupy po zakończeniu zawodów.

2. Kopia zapasowa:

- Regularne tworzenie kopii zapasowych bazy danych (np. codziennie lub co tydzień) w celu zabezpieczenia przed utratą danych.

3. Optymalizacja:

- Monitorowanie wydajności bazy danych i optymalizacja zapytań SQL.
- Tworzenie indeksów na kolumnach często używanych w zapytaniach (np. NrZawodnika w tabeli Rekord100m).

4. Bezpieczeństwo:

- Ograniczenie dostępu do bazy danych tylko do uprawnionych użytkowników.
- Regularne przeglądanie uprawnień użytkowników.

Lista tabel wraz z opisem

1. OsobaSportowa (encja nadrzędna)

- **Opis:** Przechowuje informacje o osobach związanych ze sportem (zarówno zawodnikach, jak i kibicach).
- **Kolumny:**
 - IdOsobaSportowa (PK): Unikalny identyfikator osoby.
 - Imie: Imię osoby.
 - Nazwisko: Nazwisko osoby.
 - DataUrodzenia: Data urodzenia osoby.
 - Status1: Status osoby (np. "Aktywny", "Nieaktywny").
 - DataRejestracji: Data rejestracji osoby w systemie.

2. Zawodnicy (encja podrzędna)

- **Opis:** Przechowuje informacje o zawodnikach, którzy są powiązani z osobami sportowymi.
- **Kolumny:**
 - IdZawodnicy (PK): Unikalny identyfikator zawodnika.
 - OsobaSportowaId (FK): Powiązanie z tabelą OsobaSportowa.
 - GlownaDyscyplina (FK): Główna dyscyplina sportowa zawodnika (powiązanie z tabelą Dyscyplina).
 - PESEL: Numer PESEL zawodnika.

3. Kibice (encja podrzędna)

- **Opis:** Przechowuje informacje o kibicach, którzy są powiązani z osobami sportowymi.
- **Kolumny:**
 - IdKibice (PK): Unikalny identyfikator kibica.
 - OsobaSportowaId (FK): Powiązanie z tabelą OsobaSportowa.
 - NrStowarzyszenia (FK): Numer stowarzyszenia kibiców (powiązanie z tabelą Stowarzyszenia).

4. Zawody

- **Opis:** Przechowuje informacje o zawodach sportowych.
- **Kolumny:**
 - IdZawody (PK): Unikalny identyfikator zawodów.
 - Nazwa: Nazwa zawodów.
 - Data: Data odbycia się zawodów.
 - Miejsce (FK): Miejsce zawodów (powiązanie z tabelą Miejsce).

5. ZawodnicyZawody

- **Opis:** Przechowuje informacje o udziale zawodników w zawodach.
- **Kolumny:**
 - IdZawodnicyZawody (PK): Unikalny identyfikator rekordu.
 - NrZawodnika (FK): Identyfikator zawodnika (powiązanie z tabelą Zawodnicy).
 - IdZawodow (FK): Identyfikator zawodów (powiązanie z tabelą Zawody).

6. Dyscyplina

- **Opis:** Przechowuje informacje o dyscyplinach sportowych.
- **Kolumny:**
 - IdDyscyplina (PK): Unikalny identyfikator dyscypliny.
 - Nazwa: Nazwa dyscypliny.

7. Miejsce

- **Opis:** Przechowuje informacje o miejscach, w których odbywają się zawody.
- **Kolumny:**
 - IdMiejsce (PK): Unikalny identyfikator miejsca.
 - Nazwa: Nazwa miejsca.
 - Miejscowosc: Miejscowość, w której znajduje się miejsce.

8. Trofea

- **Opis:** Przechowuje informacje o trofeach zdobytych przez zawodników.

- **Kolumny:**
 - IdTrofea (PK): Unikalny identyfikator trofeum.
 - NrZawodnika (FK): Identyfikator zawodnika (powiązanie z tabelą Zawodnicy).
 - ZdobyteTrofeum: Nazwa zdobytego trofeum.
 - IdZawodow (FK): Identyfikator zawodów (powiązanie z tabelą Zawody).
 - NrDyscyplinySportowej (FK): Identyfikator dyscypliny (powiązanie z tabelą Dyscyplina).

9. Stowarzyszenia

- **Opis:** Przechowuje informacje o stowarzyszeniach kibiców.
- **Kolumny:**
 - IdStowarzyszenia (PK): Unikalny identyfikator stowarzyszenia.
 - Nazwa: Nazwa stowarzyszenia.

10. KibicZawody

- **Opis:** Przechowuje informacje o zakupie biletów na zawody przez kibiców.
- **Kolumny:**
 - IdKibicZawody (PK): Unikalny identyfikator rekordu.
 - NrZawodow (FK): Identyfikator zawodów (powiązanie z tabelą Zawody).
 - Kibic (FK): Identyfikator kibica (powiązanie z tabelą Kibice).
 - CenaBiletu: Cena biletu.

11. Pamiatki

- **Opis:** Przechowuje informacje o pamiątkach dostępnych dla kibiców.
- **Kolumny:**
 - IdPamiatki (PK): Unikalny identyfikator pamiątki.
 - Nazwa: Nazwa pamiątki.
 - Cena: Cena pamiątki.
 - TypPamiatki (FK): Typ pamiątki (powiązanie z tabelą TypyPamiatek).
 - CzyDostepneWszedzie: Flaga określająca dostępność pamiątki (1 – dostępna wszędzie, 0 – niedostępna).

12. **TypyPamiatek**

- **Opis:** Przechowuje informacje o typach pamiątek.
- **Kolumny:**
 - IdTypyPamiatek (PK): Unikalny identyfikator typu pamiątki.
 - Nazwa: Nazwa typu pamiątki.
 - Marza: Marża na pamiątkę.

13. **Zakupy**

- **Opis:** Przechowuje informacje o zakupach pamiątek przez kibiców.
- **Kolumny:**
 - IdZakupy (PK): Unikalny identyfikator zakupu.
 - NrKlienta (FK): Identyfikator kibica (powiązanie z tabelą Kibice).
 - NrPamiatki (FK): Identyfikator pamiątki (powiązanie z tabelą Pamiatki).

- IdZawodow (FK): Identyfikator zawodów (powiązanie z tabelą Zawody).

14. **Sponsorzy**

- **Opis:** Przechowuje informacje o sponsorach zawodów.
- **Kolumny:**
 - IdSponsorzy (PK): Unikalny identyfikator sponsora.
 - Nazwa: Nazwa sponsora.

15. **Sponsoring**

- **Opis:** Przechowuje informacje o sponsoringu zawodów przez sponsorów.
- **Kolumny:**
 - IdSponsoring (PK): Unikalny identyfikator sponsoringu.
 - IdSponsora (FK): Identyfikator sponsora (powiązanie z tabelą Sponsorzy).
 - Zawody (FK): Identyfikator zawodów (powiązanie z tabelą Zawody).
 - Kwota: Kwota sponsoringu.

16. **Rekord100m**

- **Opis:** Przechowuje informacje o rekordach w biegu na 100 metrów.
- **Kolumny:**
 - IdRekord100m (PK): Unikalny identyfikator rekordu.
 - NrZawodnika (FK): Identyfikator zawodnika (powiązanie z tabelą Zawodnicy).
 - Czas_s: Czas biegu w sekundach.

17. **KonkursyDlaKibicow**

- **Opis:** Przechowuje informacje o konkursach organizowanych dla kibiców.
- **Kolumny:**
 - IdKonkursyDlaKibicow (PK): Unikalny identyfikator konkursu.
 - Nazwa: Nazwa konkursu.
 - Zwyciezca (FK): Identyfikator zwycięzcy (powiązanie z tabelą Kibice).
 - NrNagrody (FK): Identyfikator nagrody (powiązanie z tabelą Pamiatki).

Widoki:

1. **Widok_ZawodnicyTrofea**

Cel: Prezentuje informacje o zawodnikach, ich zdobytych trofeach, wydarzeniach, w których uczestniczyli, oraz dyscyplinach sportowych.

Opis: Ten widok łączy dane z tabel Trofea, Zawodnicy, Zawody oraz Dyscyplina, aby pokazać pełne informacje o zawodnikach i ich osiągnięciach. Dla każdego zawodnika widok zawiera identyfikator, imię, nazwisko, zdobyte trofeum, nazwę zawodów oraz dyscyplinę sportową.

2. **Widok_ZawodyMiejsca**

Cel: Łączy informacje o wydarzeniach sportowych z danymi o miejscach, w których się odbywają.

Opis: Ten widok łączy tabele Zawody i Miejsce, aby pokazać szczegóły dotyczące miejsca organizacji zawodów.

Dla każdego wydarzenia widok zawiera identyfikator zawodów, nazwę, datę, nazwę miejsca oraz miejscowość.

3. Widok_KibiceStowarzyszenia

Cel: Prezentuje informacje o kibicach oraz nazwach stowarzyszeń, do których należą (jeśli są przypisani).

Opis: Ten widok łączy tabele Kibice i Stowarzyszenia, aby pokazać, do jakiego stowarzyszenia należy dany kibic.

W przypadku braku przypisania do stowarzyszenia, kolumna NazwaStowarzyszenia będzie miała wartość NULL.

4. Widok_KibicZawody

Cel: Pokazuje informacje o uczestnictwie kibiców w wydarzeniach sportowych (wraz z ceną biletu).

Opis: Ten widok łączy tabele KibicZawody i Kibice, aby pokazać, którzy kibice uczestniczyli w jakich zawodach oraz ile zapłacili za bilety. Dla każdej transakcji widok zawiera identyfikator, identyfikator zawodów, imię i nazwisko kibica oraz cenę biletu.

5. Widok_ZakupyPamiatki

Cel: Łączy dane o zakupach pamiątek przez kibiców z informacjami o wydarzeniach, podczas których dokonano zakupu.

Opis: Ten widok łączy tabele Zakupy, Kibice, Pamiatki oraz Zawody, aby pokazać, jakie pamiątki zostały zakupione przez kibiców podczas konkretnych zawodów. Dla każdego zakupu widok zawiera identyfikator zakupu, imię i nazwisko kibica, nazwę pamiątki, cenę oraz nazwę zawodów.

6. Widok_Sponsoring

Cel: Prezentuje informacje o sponsoringu – łączy dane o sponsorach oraz wydarzeniach, które sponsorują.

Opis: Ten widok łączy tabele Sponsoring, Sponsorzy oraz Zawody, aby pokazać, którzy sponsorzy wsparli jakie wydarzenia oraz jaka była kwota sponsoringu. Dla każdego sponsoringu widok zawiera identyfikator, nazwę sponsora, nazwę zawodów oraz kwotę.

7. Widok_KonkursyKibicow

Cel: Łączy informacje o konkursach dla kibiców z danymi o zwycięzcach oraz nagrodach.

Opis: Ten widok łączy tabele KonkursyDlaKibicow, Kibice oraz Pamiatki, aby pokazać, jakie konkursy zostały zorganizowane, kto je wygrał oraz jakie nagrody zostały przyznane. Dla każdego konkursu widok zawiera identyfikator, nazwę konkursu, imię i nazwisko zwycięzcy oraz nazwę nagrody.

Procedury składowane:

1. Procedura DodajOsobeSportowa

Procedura DodajOsobeSportowa służy do dodawania nowych osób sportowych do systemu. Osoby sportowe mogą być zarówno zawodnikami, jak i kibicami. Procedura sprawdza, czy podana data urodzenia jest w przeszłości, aby uniknąć błędnych danych. Jeśli data urodzenia jest poprawna, nowa osoba sportowa jest dodawana do tabeli OsobaSportowa.

2. Procedura UsunOsobeSportowa

Procedura UsunOsobeSportowa umożliwia usunięcie osoby sportowej z systemu na podstawie jej identyfikatora.

Przed usunięciem procedura sprawdza, czy osoba sportowa o podanym ID istnieje. Jeśli osoba istnieje, zostaje usunięta z tabeli OsobaSportowa. Dzięki zastosowaniu kluczy obcych z opcją ON DELETE CASCADE, usunięcie osoby sportowej powoduje również automatyczne usunięcie powiązanych rekordów w tabelach Zawodnicy i Kibice.

3. Procedura AktualizujStatusOsobySportowej

Procedura AktualizujStatusOsobySportowej pozwala na zmianę statusu osoby sportowej (np. z "Aktywny" na "Nieaktywny"). Przed aktualizacją procedura sprawdza, czy osoba sportowa o podanym ID istnieje. Jeśli istnieje, jej status zostaje zaktualizowany w tabeli OsobaSportowa.

4. Procedura PobierzZawodnikowZDyscypliny

Procedura PobierzZawodnikowZDyscypliny zwraca listę zawodników przypisanych do określonej dyscypliny sportowej. Przed zwróceniem wyników procedura sprawdza, czy dyscyplina o podanym ID istnieje. Jeśli dyscyplina istnieje, procedura zwraca informacje o zawodnikach, w tym ich imię, nazwisko oraz nazwę dyscypliny.

5. Procedura DodajZawody

Procedura DodajZawody służy do dodawania nowych zawodów sportowych do systemu. Przyjmuje parametry takie jak nazwa zawodów, data ich przeprowadzenia oraz identyfikator miejsca. Procedura sprawdza dwa kluczowe warunki: Czy data zawodów jest w przeszłości (aby uniknąć planowania wydarzeń na przyszłe daty). Czy miejsce zawodów istnieje w tabeli Miejsce (poprzez weryfikację klucza obcego).

Funkcje:

1. Funkcja dbo.F_RankingZawodnika

Cel: Oblicza ranking zawodnika w biegu na 100 metrów na podstawie jego rekordu czasowego.

Opis: Funkcja przyjmuje identyfikator zawodnika (@IdZawodnika) i zwraca jego pozycję w rankingu.

Ranking jest obliczany na podstawie czasu zawodnika w porównaniu do czasów innych zawodników w tabeli Rekord100m. Im mniejszy czas, tym wyższa pozycja w rankingu. Jeśli zawodnik nie ma rekordu w tabeli Rekord100m, funkcja zwraca NULL.

2. Funkcja dbo.F_SpodsumowanieWydatkowKibica

Cel: Zwraca podsumowanie wydatków kibica na bilety oraz pamiątki.

Opis: Funkcja przyjmuje identyfikator kibica (@IdKibice) i zwraca tabelę z trzema kolumnami:

TotalTicketCost: Łączna kwota wydana na bilety przez kibica, TotalSouvenirCost: Łączna kwota wydana na pamiątki przez kibica, OverallCost: Suma wydatków na bilety i pamiątki. Funkcja korzysta z tabel KibicZawody (do obliczenia kosztów biletów) oraz Zakupy i Pamiatki (do obliczenia kosztów pamiątek). Jeśli kibic nie ma wydatków, funkcja zwraca 0 w odpowiednich kolumnach.

3. Funkcja dbo.F_DyscyplinaPerformance

Cel: Analizuje rekordy zawodników w danej dyscyplinie sportowej.

Opis: Funkcja przyjmuje identyfikator dyscypliny (@IdDyscypliny) i zwraca zestawienie rekordów zawodników, którzy mają tę dyscyplinę jako główną.

Wynik zawiera: LiczbaRekordow: Liczba zarejestrowanych rekordów w danej dyscyplinie, NajlepszyCzas: Najlepszy (najmniejszy) czas wśród rekordów, NajgorszyCzas: Najgorszy (największy) czas wśród rekordów, SredniCzas: Średni czas wśród rekordów.

Funkcja korzysta z tabel Rekord100m i Zawodnicy, aby połączyć rekordy z zawodnikami i ich głównymi dyscyplinami.

Wyzwalacze:

1. Ustawia status osoby sportowej na „Aktywny” po dodaniu jej jako zawodnika

Po dodaniu rekordu do tabeli *Zawodnicy*, wyzwalacz automatycznie ustawia w tabeli *OsobaSportowa* kolumnę *Status1* na „Aktywny”.

Dzięki temu, gdy osoba staje się zawodnikiem, jej status jest natychmiast aktualizowany.

2. Walidacja formatu PESEL w tabeli Zawodnicy

Wyzwalacz weryfikuje, czy numer PESEL zawiera dokładnie 11 cyfr.

Jeśli format jest nieprawidłowy, operacja wstawiania lub aktualizacji zostaje przerwana, co zapewnia poprawność danych.

3. Ochrona przed usunięciem osoby sportowej aktywnie uczestniczącej w zawodach

Wyzwalacz typu *INSTEAD OF DELETE* blokuje usunięcie osoby z tabeli *OsobaSportowa*, jeśli jest powiązana z rekordami w tabelach *Zawodnicy* i *ZawodnicyZawody*. Chroni to cenne dane dotyczące aktywności sportowej.

4. Sprawdzenie dostępności miejsca (uniknięcie podwójnej rezerwacji)

Po dodaniu rekordu do tabeli *Zawody*, wyzwalacz sprawdza, czy w tym samym dniu oraz miejscu nie jest już zaplanowane inne wydarzenie. W przypadku konfliktu operacja jest wycofywana.

5. Ogłoszenie nowego rekordu w biegu na 100m

Wyzwalacz monitoruje dodawane wyniki w tabeli *Rekord100m*. Jeśli wstawiony czas jest najniższym spośród wszystkich zapisanych, wyświetla komunikat informujący o ustanowieniu nowego rekordu.

Indeksy:

1. idx_osoba_sportowa

- Przyspiesza wyszukiwanie rekordów w tabeli *OsobaSportowa* na podstawie kolumny *IdOsobaSportowa*, co usprawnia pobieranie danych i sprawdzanie statusu danej osoby.

2. idx_glowna_dyscyplina

- Optymalizuje zapytania wyszukujące zawodników według ich głównej dyscypliny (*GlownaDyscyplina*). Jest przydatny przy analizach porównawczych i przydzielaniu zawodników do wydarzeń.

3. idx_nr_zawodnika

- Umożliwia szybsze wyszukiwanie informacji w tabeli ZawodnicyZawody poprzez filtrowanie według numeru zawodnika (NrZawodnika), co pomaga w raportowaniu osiągnięć i uczestnictwa.

4. idx_id_zawodow

- Usprawnia wyszukiwanie zawodów po ich unikalnym identyfikatorze (IdZawody), co jest kluczowe przy pobieraniu szczegółów wydarzeń, list uczestników oraz wyników.

5. idx_kibic

- Przyspiesza operacje wyszukiwania w tabeli Kibice poprzez indeksowanie kolumny OsobaSportowald. Ułatwia to łączenie danych o kibicach z innymi tabelami, np. przy obsłudze stowarzyszeń czy zakupów biletów.

6. idx_pamiatki

- Umożliwia szybkie filtrowanie pamiątek według typu (TypPamiatki), co przyspiesza wyszukiwanie dostępnych gadżetów i pomaga w zarządzaniu asortymentem oraz analizie sprzedaży.

Typowe zapytania:

1. Pobierz wszystkich zawodników wraz z ich danymi osobowymi i dyscypliną

Opis:

To zapytanie zwraca listę wszystkich zawodników wraz z ich danymi osobowymi (imię, nazwisko) oraz nazwą dyscypliny, którą uprawiają.

2. Pobierz wszystkie zawody wraz z miejscem ich odbycia

Opis:

To zapytanie zwraca listę wszystkich zawodów wraz z nazwą miejsca, w którym się odbywają, oraz datą zawodów.

3. Pobierz wszystkie trofea zdobyte przez zawodników

Opis:

To zapytanie zwraca listę wszystkich trofeów zdobytych przez zawodników, wraz z ich danymi osobowymi (imię, nazwisko) oraz nazwą dyscypliny, w której zdobyli trofeum.

4. Pobierz zawodników u18 (urodzonych po 2007 roku)

Opis:

To zapytanie zwraca listę zawodników, którzy są niepełnoletni lub mają równe 18 lat (urodzeni po 2007 roku).

5. Pobierz liczbę zawodników w każdej dyscyplinie

Opis:

To zapytanie zwraca liczbę zawodników uprawiających każdą dyscyplinę sportową.

Diagramy relacji:

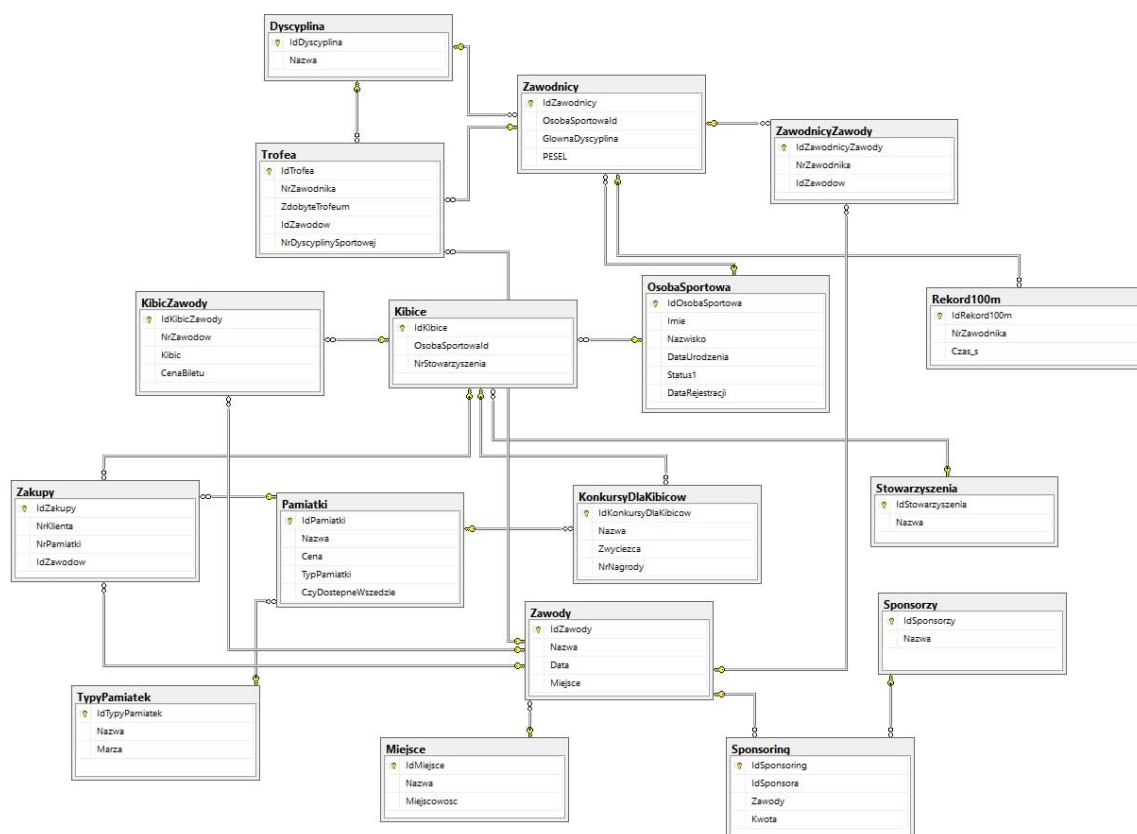
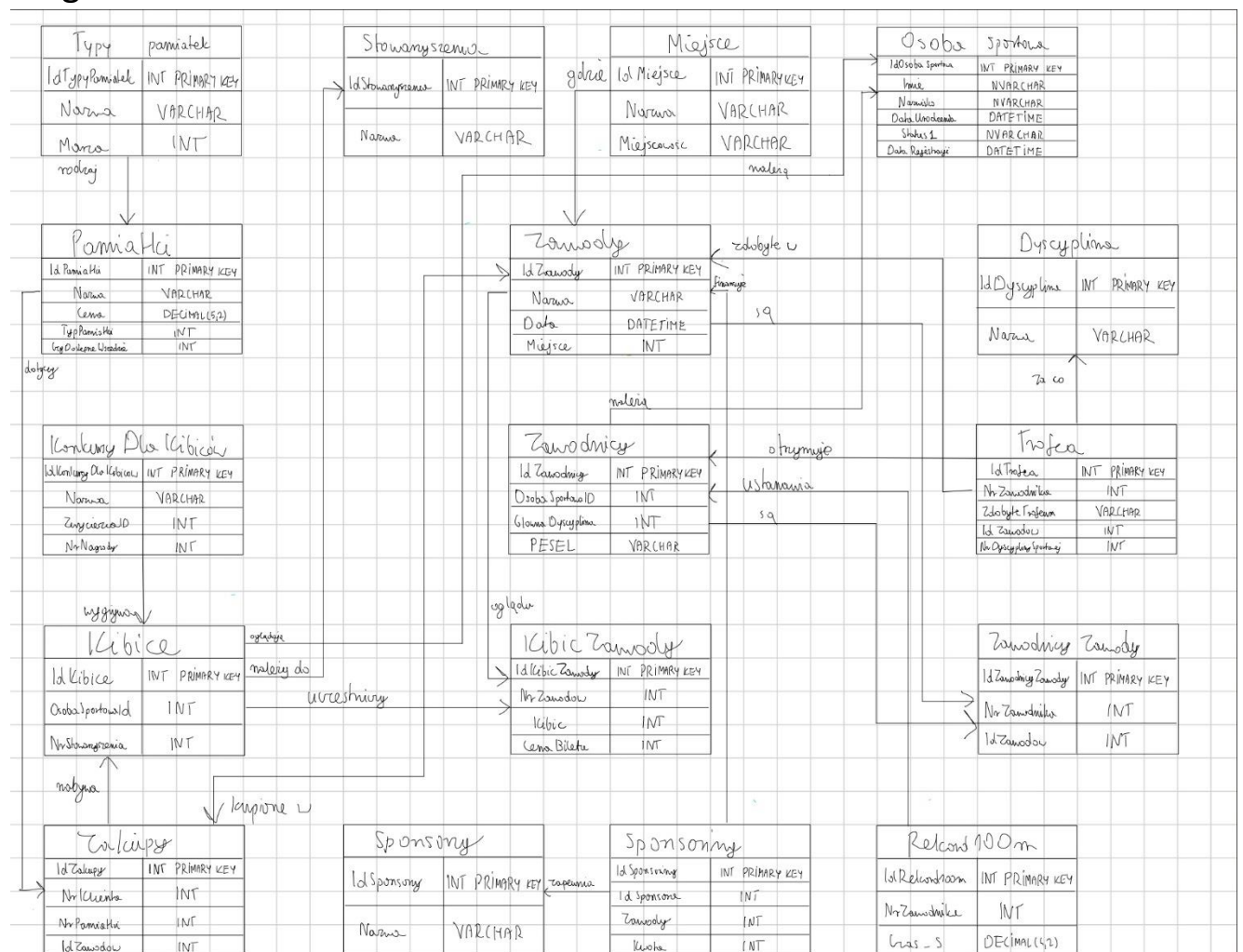


Diagram ER:



Dodatkowe więzy integralności danych:

1.Ograniczenia CHECK:

- Sprawdź, czy DataUrodzenia jest wcześniejsza niż bieżąca data: `CHECK (DataUrodzenia < GETDATE());`
- Sprawdź, czy Data zawodów jest w przeszłości: `CHECK (Data < GETDATE());`
- Sprawdź, czy cena biletu jest większa niż 0:

CHECK (CenaBiletu > 0);

2. Unikalne rekordy:

--UNIQUE dla kolumny PESEL - każdy zawodnik ma unikalny numer PESEL:

--UNIQUE dla kolumny Nazwa - nazwy dyscyplin muszą być unikalne:

--UNIQUE dla kolumny Nazwa - nazwy stowarzyszeń muszą być unikalne:

3. Ograniczenie NOT NULL: Wymusza obecność wartości w wielu kluczowych polach:

1. **OsobaSportowa** -> IdOsobaSportowa

2. **OsobaSportowa** -> Imie

3. **OsobaSportowa** -> Nazwisko

4. **OsobaSportowa** -> DataUrodzenia

5. **OsobaSportowa** -> Status1

6. **Zawodnicy** -> IdZawodnicy

7. **Zawodnicy** -> OsobaSportowaId

8. **Zawodnicy** -> GlownaDyscyplina

9. **Zawodnicy** -> PESEL

10. **Kibice** -> IdKibice

11. **Kibice** -> OsobaSportowaId

12. **Zawody** -> IdZawody

13. **Zawody** -> Nazwa

14. **Zawody** -> Data

15. **Zawody** -> Miejsce

16. **ZawodnicyZawody** -> IdZawodnicyZawody

17. **ZawodnicyZawody** -> NrZawodnika
18. **ZawodnicyZawody** -> IdZawodow
19. **Dyscyplina** -> IdDyscyplina
20. **Dyscyplina** -> Nazwa
21. **Miejsce** -> IdMiejsce
22. **Miejsce** -> Nazwa
23. **Miejsce** -> Miejscowosc
24. **Trofea** -> IdTrofea
25. **Trofea** -> NrZawodnika
26. **Trofea** -> ZdobyteTrofeum
27. **Trofea** -> IdZawodow
28. **Trofea** -> NrDyscyplinySportowej
29. **Stowarzyszenia** -> IdStowarzyszenia
30. **Stowarzyszenia** -> Nazwa
31. **KibicZawody** -> IdKibicZawody
32. **KibicZawody** -> NrZawodow
33. **KibicZawody** -> Kibic
34. **KibicZawody** -> CenaBiletu
35. **Pamiatki** -> IdPamiatki
36. **Pamiatki** -> Nazwa
37. **Pamiatki** -> Cena
38. **Pamiatki** -> TypPamiatki
39. **Pamiatki** -> CzyDostepneWszedzie
40. **TypyPamiatek** -> IdTypyPamiatek

- 41. **TypyPamiatek** -> Nazwa
- 42. **TypyPamiatek** -> Marza
- 43. **Zakupy** -> IdZakupy
- 44. **Zakupy** -> NrKlienta
- 45. **Zakupy** -> NrPamiatki
- 46. **Zakupy** -> IdZawodow
- 47. **Sponsorzy** -> IdSponsorzy
- 48. **Sponsorzy** -> Nazwa
- 49. **Sponsoring** -> IdSponsoring
- 50. **Sponsoring** -> IdSponsora
- 51. **Sponsoring** -> Zawody
- 52. **Sponsoring** -> Kwota
- 53. **Rekord100m** -> IdRekord100m
- 54. **Rekord100m** -> NrZawodnika
- 55. **Rekord100m** -> Czas_s
- 56. **KonkursyDlaKibicow** -> IdKonkursyDlaKibicow
- 57. **KonkursyDlaKibicow** -> Nazwa
- 58. **KonkursyDlaKibicow** -> Zwyciezca
- 59. **KonkursyDlaKibicow** -> NrNagrody

4. Kaskadowe operacja usuwania ON DELETE CASCADE zapewnia, że po usunięciu powiązanych danych usunięte zostaną również wpisy zależne:

- 1. Usunięcie osoby sportowej (OsobaSportowa)

Efekt: Usunięcie osoby sportowej powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabelach Zawodnicy i Kibice.

2. Usunięcie zawodnika (Zawodnicy)

Efekt: Usunięcie zawodnika powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabelach ZawodnicyZawody, Trofea i Rekord100m.

3. Usunięcie kibica (Kibice)

Efekt: Usunięcie kibica powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabelach KibicZawody, Zakupy i KonkursyDlaKibicow.

4. Usunięcie zawodów (Zawody)

Efekt: Usunięcie zawodów powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabelach ZawodnicyZawody, Trofea, KibicZawody, Zakupy i Sponsoring.

5. Usunięcie dyscypliny (Dyscyplina)

Efekt: Usunięcie dyscypliny powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabeli Trofea.

6. Usunięcie stowarzyszenia (Stowarzyszenia)

Efekt: Usunięcie stowarzyszenia powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabeli Kibice.

7. Usunięcie pamiątki (Pamiatki)

Efekt: Usunięcie pamiątki powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabelach Zakupy i KonkursyDlaKibicow.

8. Usunięcie sponsora (Sponsorzy)

Efekt: Usunięcie sponsora powoduje automatyczne usunięcie wszystkich powiązanych rekordów w tabeli Sponsoring.

Skrypt tworzący bazę danych:

-- Usuń bazę danych, jeśli istnieje

IF DB_ID('ZawodySportowe') IS NOT NULL

DROP DATABASE ZawodySportowe;

GO

-- Utwórz bazę danych

CREATE DATABASE ZawodySportowe;

GO

-- Użyj bazy danych ZawodySportowe

USE ZawodySportowe;

GO

-- Tabela OsobaSportowa (encja nadrzędna)

CREATE TABLE OsobaSportowa (

IdOsobaSportowa INT PRIMARY KEY,


```
Imie NVARCHAR(50) NOT NULL,  
Nazwisko NVARCHAR(50) NOT NULL,  
DataUrodzenia DATETIME NOT NULL,  
Status1 NVARCHAR(50) NOT NULL,  
DataRejestracji DATETIME DEFAULT GETDATE()  
);
```

-- Tabela Zawodnicy (encja podrzędna)

```
CREATE TABLE Zawodnicy (  
    IdZawodnicy INT PRIMARY KEY,  
    OsobaSportowald INT NOT NULL,  
    GlownaDyscyplina INT NOT NULL,  
    PESEL VARCHAR(11) NOT NULL,  
    CONSTRAINT FK_Zawodnicy_OsobaSportowa FOREIGN KEY  
(OsobaSportowald) REFERENCES OsobaSportowa(IdOsobaSportowa)  
ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

-- Tabela Kibice (encja podrzędna)

```
CREATE TABLE Kibice (  
    IdKibice INT PRIMARY KEY,  
    OsobaSportowald INT NOT NULL,  
    NrStowarzyszenia INT NULL,
```

```
CONSTRAINT FK_Kibice_OsobaSportowa FOREIGN KEY  
(OsobaSportowaId) REFERENCES OsobaSportowa(IdOsobaSportowa)  
ON DELETE CASCADE  
  
ON UPDATE CASCADE  
);
```

-- Tabela Zawody

```
CREATE TABLE Zawody (  
    IdZawody INT PRIMARY KEY,  
    Nazwa VARCHAR(46) NOT NULL,  
    Data DATETIME NOT NULL,  
    Miejsce INT NOT NULL  
);
```

-- Tabela ZawodnicyZawody

```
CREATE TABLE ZawodnicyZawody (  
    IdZawodnicyZawody INT PRIMARY KEY,  
    NrZawodnika INT NOT NULL,  
    IdZawodow INT NOT NULL,  
  
    CONSTRAINT FK_ZawodnicyZawody_Zawodnicy FOREIGN KEY  
(NrZawodnika) REFERENCES Zawodnicy(IdZawodnicy) ON DELETE  
CASCADE  
  
ON UPDATE CASCADE,  
  
    CONSTRAINT FK_ZawodnicyZawody_Zawody FOREIGN KEY  
(IdZawodow) REFERENCES Zawody(IdZawody)
```

);

-- Tabela Dyscyplina

CREATE TABLE Dyscyplina (

IdDyscyplina INT PRIMARY KEY,

Nazwa VARCHAR(14) NOT NULL

);

-- Tabela Miejsce

CREATE TABLE Miejsce (

IdMiejsce INT PRIMARY KEY,

Nazwa VARCHAR(29) NOT NULL,

Miejscowosc VARCHAR(10) NOT NULL

);

-- Tabela Trofea

CREATE TABLE Trofea (

IdTrofea INT PRIMARY KEY,

NrZawodnika INT NOT NULL,

ZdobyteTrofeum VARCHAR(34) NOT NULL,

IdZawodow INT NOT NULL,

NrDyscyplinySportowej INT NOT NULL,

CONSTRAINT FK_Trofea_Zawodnicy FOREIGN KEY (NrZawodnika)
REFERENCES Zawodnicy(IdZawodnicy) ON DELETE CASCADE

```
ON UPDATE CASCADE,  
  
CONSTRAINT FK_Trofea_Zawody FOREIGN KEY (IdZawodow)  
REFERENCES Zawody(IdZawody),  
  
CONSTRAINT FK_Trofea_Dyscyplina FOREIGN KEY  
(NrDyscyplinySportowej) REFERENCES Dyscyplina(IdDyscyplina)  
);
```

-- Tabela Stowarzyszenia

```
CREATE TABLE Stowarzyszenia (  
  
    IdStowarzyszenia INT PRIMARY KEY,  
  
    Nazwa VARCHAR(46) NOT NULL  
  
);
```

-- Tabela KibicZawody

```
CREATE TABLE KibicZawody (  
  
    IdKibicZawody INT PRIMARY KEY,  
  
    NrZawodow INT NOT NULL,  
  
    Kibic INT NOT NULL,  
  
    CenaBiletu INT NOT NULL,  
  
    CONSTRAINT FK_KibicZawody_Zawody FOREIGN KEY (NrZawodow)  
REFERENCES Zawody(IdZawody),  
  
    CONSTRAINT FK_KibicZawody_Kibice FOREIGN KEY (Kibic)  
REFERENCES Kibice(IdKibice) ON DELETE CASCADE  
  
ON UPDATE CASCADE,  
  
);
```

-- Tabela Pamiatki

```
CREATE TABLE Pamiatki (  
    IdPamiatki INT PRIMARY KEY,  
    Nazwa VARCHAR(22) NOT NULL,  
    Cena DECIMAL(5,2) NOT NULL,  
    TypPamiatki INT NOT NULL,  
    CzyDostepneWszedzie INT NOT NULL  
);
```

-- Tabela TypyPamiatek

```
CREATE TABLE TypyPamiatek (  
    IdTypyPamiatek INT PRIMARY KEY,  
    Nazwa VARCHAR(25) NOT NULL,  
    Marza INT NOT NULL  
);
```

-- Tabela Zakupy

```
CREATE TABLE Zakupy (  
    IdZakupy INT PRIMARY KEY,  
    NrKlienta INT NOT NULL,  
    NrPamiatki INT NOT NULL,  
    IdZawodow INT NOT NULL,
```

```
    CONSTRAINT FK_Zakupy_Kibice FOREIGN KEY (NrKlienta)
REFERENCES Kibice(IdKibice) ON DELETE CASCADE
ON UPDATE CASCADE,

    CONSTRAINT FK_Zakupy_Pamiatki FOREIGN KEY (NrPamiatki)
REFERENCES Pamiatki(IdPamiatki),

    CONSTRAINT FK_Zakupy_Zawody FOREIGN KEY (IdZawodow)
REFERENCES Zawody(IdZawody)
);
```

-- Tabela Sponsorzy

```
CREATE TABLE Sponsorzy (
    IdSponsorzy INT PRIMARY KEY,
    Nazwa VARCHAR(30) NOT NULL
);
```

-- Tabela Sponsoring

```
CREATE TABLE Sponsoring (
    IdSponsoring INT PRIMARY KEY,
    IdSponsora INT NOT NULL,
    Zawody INT NOT NULL,
    Kwota INT NOT NULL,

    CONSTRAINT FK_Sponsoring_Sponsorzy FOREIGN KEY (IdSponsora)
REFERENCES Sponsorzy(IdSponsorzy),

    CONSTRAINT FK_Sponsoring_Zawody FOREIGN KEY (Zawody)
REFERENCES Zawody(IdZawody)
```

);

-- Tabela Rekord100m

```
CREATE TABLE Rekord100m (  
    IdRekord100m INT PRIMARY KEY,  
    NrZawodnika INT NOT NULL,  
    Czas_s DECIMAL(4,2) NOT NULL,  
    CONSTRAINT FK_Rekord100m_Zawodnicy FOREIGN KEY  
(NrZawodnika) REFERENCES Zawodnicy(IdZawodnicy) ON DELETE  
CASCADE  
ON UPDATE CASCADE,  
);
```

-- Tabela KonkursyDlaKibicow

```
CREATE TABLE KonkursyDlaKibicow (  
    IdKonkursyDlaKibicow INT PRIMARY KEY,  
    Nazwa VARCHAR(50) NOT NULL,  
    Zwyciezca INT NOT NULL,  
    NrNagrody INT NOT NULL,  
    CONSTRAINT FK_KonkursyDlaKibicow_Kibice FOREIGN KEY  
(Zwyciezca) REFERENCES Kibice(IdKibice) ON DELETE CASCADE  
ON UPDATE CASCADE,  
    CONSTRAINT FK_KonkursyDlaKibicow_Pamiatki FOREIGN KEY  
(NrNagrody) REFERENCES Pamiatki(IdPamiatki)  
);
```

-- Dodaj klucze obce do tabeli Zawodnicy

ALTER TABLE Zawodnicy

ADD CONSTRAINT FK_Zawodnicy_Dyscyplina FOREIGN KEY
(GlownaDyscyplina) REFERENCES Dyscyplina(IdDyscyplina);

-- Dodaj klucze obce do tabeli Zawody

ALTER TABLE Zawody

ADD CONSTRAINT FK_Zawody_Miejsce FOREIGN KEY (Miejsce)
REFERENCES Miejsce(IdMiejsce);

-- Dodaj klucze obce do tabeli Kibice

ALTER TABLE Kibice

ADD CONSTRAINT FK_Kibice_Stowarzyszenia FOREIGN KEY
(NrStowarzyszenia) REFERENCES Stowarzyszenia(IdStowarzyszenia);

-- Dodaj klucze obce do tabeli Pamiatki

ALTER TABLE Pamiatki

ADD CONSTRAINT FK_Pamiatki_TypyPamiatek FOREIGN KEY
(TypPamiatki) REFERENCES TypyPamiatek(IdTypyPamiatek);

--Sprawdź, czy DataUrodzenia jest wcześniejsza niż bieżąca data:

ALTER TABLE OsobaSportowa

ADD CONSTRAINT CHK_OsobaSportowa_DataUrodzenia CHECK
(DataUrodzenia < GETDATE());

--Sprawdź, czy Data zawodów jest w przeszłości:


```
ALTER TABLE Zawody
```

```
ADD CONSTRAINT CHK_Zawody_Data CHECK (Data < GETDATE());
```

```
--Sprawdź, czy cena biletu jest większa niż 0
```

```
ALTER TABLE KibicZawody
```

```
ADD CONSTRAINT CHK_KibicZawody_CenaBiletu CHECK (CenaBiletu > 0);
```

```
--Usuń zawodnika, jeśli zostanie usunięta powiązana osoba sportowa:
```

```
ALTER TABLE Zawodnicy
```

```
DROP CONSTRAINT FK_Zawodnicy_OsobaSportowa;
```

```
--Usuń kibica, jeśli zostanie usunięta powiązana osoba sportowa:
```

```
ALTER TABLE Kibice
```

```
DROP CONSTRAINT FK_Kibice_OsobaSportowa;
```

```
ALTER TABLE Kibice
```

```
ADD CONSTRAINT FK_Kibice_OsobaSportowa
```

```
FOREIGN KEY (OsobaSportowaId) REFERENCES  
OsobaSportowa(IdOsobaSportowa)
```

```
ON DELETE CASCADE;
```

```
--
```

```
ALTER TABLE Zawodnicy
```

```
ADD CONSTRAINT FK_Zawodnicy_OsobaSportowa
```

```
FOREIGN KEY (OsobaSportowaId) REFERENCES  
OsobaSportowa(IdOsobaSportowa)
```

```
ON DELETE CASCADE;
```

--UNIQUE dla kolumny PESEL - każdy zawodnik ma unikalny numer PESEL:

```
ALTER TABLE Zawodnicy
```

```
ADD CONSTRAINT UQ_Zawodnicy_PESEL UNIQUE (PESEL);
```

--UNIQUE dla kolumny Nazwa - nazwy dyscyplin muszą być unikalne:

```
ALTER TABLE Dyscyplina
```

```
ADD CONSTRAINT UQ_Dyscyplina_Nazwa UNIQUE (Nazwa);
```

--UNIQUE dla kolumny Nazwa - nazwy stowarzyszeń muszą być unikalne:

```
ALTER TABLE Stowarzyszenia
```

```
ADD CONSTRAINT UQ_Stowarzyszenia_Nazwa UNIQUE (Nazwa);
```

```
INSERT INTO OsobaSportowa (IdOsobaSportowa, Imie, Nazwisko, DataUrodzenia, Status1, DataRejestracji) VALUES
```

```
(1, 'Jan', 'Kowalski', '1990-05-15', 'Aktywny', '2023-01-01'),
```

```
(2, 'Anna', 'Nowak', '1985-08-22', 'Aktywny', '2023-01-02'),
```

```
(3, 'Piotr', 'Wiśniewski', '1995-03-10', 'Nieaktywny', '2023-01-03'),
```

```
(4, 'Maria', 'Dąbrowska', '1992-11-30', 'Aktywny', '2023-01-04'),
```

```
(5, 'Krzysztof', 'Lewandowski', '1988-07-25', 'Aktywny', '2023-01-05'),
```

```
(6, 'Agnieszka', 'Wójcik', '1998-09-12', 'Nieaktywny', '2023-01-06'),
```

```
(7, 'Marek', 'Kamiński', '1991-04-18', 'Aktywny', '2023-01-07');
```

```
INSERT INTO Dyscyplina (IdDyscyplina, Nazwa) VALUES
```

(1, 'Biegi'),
(2, 'Pływanie'),
(3, 'Skoki'),
(4, 'Kolarstwo'),
(5, 'Tenis'),
(6, 'Piłka nożna'),
(7, 'Siatkówka');

INSERT INTO Zawodnicy (IdZawodnicy, OsobaSportowald,
GlownaDyscyplina, PESEL) VALUES

(1, 1, 1, '90051512345'),
(2, 2, 2, '85082254321'),
(3, 3, 3, '95031098765'),
(4, 4, 4, '92113045678'),
(5, 5, 5, '88072567890'),
(6, 6, 6, '98091234567'),
(7, 7, 7, '91041887654');

INSERT INTO Stowarzyszenia (IdStowarzyszenia, Nazwa) VALUES

(1, 'Stowarzyszenie Kibiców Biegów'),
(2, 'Stowarzyszenie Kibiców Pływania'),
(3, 'Stowarzyszenie Kibiców Skoków'),

(4, 'Stowarzyszenie Kibiców Kolarstwa'),
(5, 'Stowarzyszenie Kibiców Tenisa'),
(6, 'Stowarzyszenie Kibiców Piłki Nożnej'),
(7, 'Stowarzyszenie Kibiców Siatkówki');

INSERT INTO Kibice (IdKibice, OsobaSportowaId, NrStowarzyszenia)
VALUES

(1, 1, 1),
(2, 2, 2),
(3, 3, 3),
(4, 4, 4),
(5, 5, 5),
(6, 6, 6),
(7, 7, 7);

INSERT INTO Miejsce (IdMiejsce, Nazwa, Miejscowosc) VALUES

(1, 'Stadion Narodowy', 'Warszawa'),
(2, 'Arena Gdańsk', 'Gdańsk'),
(3, 'Hala Olivia', 'Gdańsk'),
(4, 'Stadion Miejski', 'Poznań'),
(5, 'Hala Torwar', 'Warszawa'),
(6, 'Stadion Śląski', 'Chorzów'),
(7, 'Hala Orbita', 'Wrocław');

```
INSERT INTO Zawody (IdZawody, Nazwa, Data, Miejsce) VALUES
(1, 'Mistrzostwa Polski w Biegach', '2024-06-01', 1),
(2, 'Mistrzostwa Europy w Pływaniu', '2024-07-15', 2),
(3, 'Puchar Świata w Skokach', '2024-08-20', 3),
(4, 'Tour de Pologne', '2024-09-10', 4),
(5, 'Wimbledon', '2024-07-01', 5),
(6, 'Liga Mistrzów', '2024-05-30', 6),
(7, 'Mistrzostwa Świata w Siatkówce', '2024-10-01', 7);
```

```
INSERT INTO ZawodnicyZawody (IdZawodnicyZawody, NrZawodnika,
IdZawodow) VALUES
(1, 1, 1),
(2, 2, 2),
(3, 3, 3),
(4, 4, 4),
(5, 5, 5),
(6, 6, 6),
(7, 7, 7);
```

```
INSERT INTO Trofea (IdTrofea, NrZawodnika, ZdobyteTrofeum,
IdZawodow, NrDyscyplinySportowej) VALUES
(1, 1, 'Złoty medal', 1, 1),
(2, 2, 'Srebrny medal', 2, 2),
(3, 3, 'Brązowy medal', 3, 3),
```

```
(4, 4, 'Złoty medal', 4, 4),  
(5, 5, 'Srebrny medal', 5, 5),  
(6, 6, 'Brązowy medal', 6, 6),  
(7, 7, 'Złoty medal', 7, 7);
```

```
INSERT INTO KibicZawody (IdKibicZawody, NrZawodow, Kibic,  
CenaBiletu) VALUES
```

```
(1, 1, 1, 100),  
(2, 2, 2, 150),  
(3, 3, 3, 200),  
(4, 4, 4, 250),  
(5, 5, 5, 300),  
(6, 6, 6, 350),  
(7, 7, 7, 400);
```

```
INSERT INTO TypyPamiatek (IdTypyPamiatek, Nazwa, Marza) VALUES
```

```
(1, 'Kubki', 10),  
(2, 'Koszulki', 20),  
(3, 'Czapki', 15),  
(4, 'Plakaty', 5),  
(5, 'Plecaki', 25),  
(6, 'Medale', 30),  
(7, 'Figurki', 35);
```

```
INSERT INTO Pamiatki (IdPamiatki, Nazwa, Cena, TypPamiatki,  
CzyDostepneWszedzie) VALUES
```

```
(1, 'Kubek', 20.00, 1, 1),  
(2, 'Koszulka', 50.00, 2, 1),  
(3, 'Czapka', 30.00, 3, 1),  
(4, 'Plakat', 15.00, 4, 0),  
(5, 'Plecak', 70.00, 5, 1),  
(6, 'Medal', 100.00, 6, 0),  
(7, 'Figurka', 120.00, 7, 0);
```

```
INSERT INTO Zakupy (IdZakupy, NrKlienta, NrPamiatki, IdZawodow)  
VALUES
```

```
(1, 1, 1, 1),  
(2, 2, 2, 2),  
(3, 3, 3, 3),  
(4, 4, 4, 4),  
(5, 5, 5, 5),  
(6, 6, 6, 6),  
(7, 7, 7, 7);
```

```
INSERT INTO Sponsorzy (IdSponsorzy, Nazwa) VALUES
```

```
(1, 'Nike'),  
(2, 'Adidas'),  
(3, 'Puma'),
```

```
(4, 'Red Bull'),  
(5, 'Coca-Cola'),  
(6, 'Pepsi'),  
(7, 'Toyota');
```

```
INSERT INTO Sponsoring (IdSponsoring, IdSponsora, Zawody, Kwota)  
VALUES
```

```
(1, 1, 1, 100000),  
(2, 2, 2, 150000),  
(3, 3, 3, 200000),  
(4, 4, 4, 250000),  
(5, 5, 5, 300000),  
(6, 6, 6, 350000),  
(7, 7, 7, 400000);
```

```
INSERT INTO Rekord100m (IdRekord100m, NrZawodnika, Czas_s)  
VALUES
```

```
(1, 1, 9.58),  
(2, 2, 10.12),  
(3, 3, 10.45),  
(4, 4, 10.78),  
(5, 5, 11.02),  
(6, 6, 11.34),  
(7, 7, 11.56);
```



```
INSERT INTO KonkursyDlaKibicow (IdKonkursyDlaKibicow, Nazwa, Zwyciezca, NrNagrody) VALUES
```

```
(1, 'Konkurs wiedzy sportowej', 1, 1),
```

```
(2, 'Konkurs plastyczny', 2, 2),
```

```
(3, 'Konkurs fotograficzny', 3, 3),
```

```
(4, 'Konkurs karaoke', 4, 4),
```

```
(5, 'Konkurs taneczny', 5, 5),
```

```
(6, 'Konkurs wiedzy o klubie', 6, 6),
```

```
(7, 'Konkurs quizowy', 7, 7);
```

--Widoki

-- Wyświetla zawodników wraz z informacjami o zdobytych trofeach, wydarzeniach oraz dyscyplinach.

```
CREATE VIEW Widok_ZawodnicyTrofea AS
```

```
SELECT
```

```
    z.IdZawodnicy,
```

```
    z.Imie,
```

```
    z.Nazwisko,
```

```
    t.ZdobyteTrofeum,
```

```
    w.Nazwa AS NazwaZawodow,
```

```
    d.Nazwa AS Dyscyplina
```

```
FROM Trofea t
```

```
JOIN Zawodnicy z ON t.NrZawodnika = z.IdZawodnicy
```

```
JOIN Zawody w ON t.IdZawodow = w.IdZawody
```

```
JOIN Dyscyplina d ON t.NrDyscyplinySportowej = d.IdDyscyplina;
```

```
GO
```

-- Łączy informacje o wydarzeniach ze szczegółami miejsc, gdzie się odbywają.

```
CREATE VIEW Widok_ZawodyMiejsca AS
```

```
SELECT
```

```
    w.IdZawody,
```

```
    w.Nazwa AS NazwaZawodow,
```

```
    w.Data,
```

```
    m.Nazwa AS NazwaMiejsca,
```

```
    m.Miejscowosc
```

```
FROM Zawody w
```

```
JOIN Miejsce m ON w.Miejsce = m.IdMiejsce;
```

```
GO
```

-- Prezentuje kibiców wraz z nazwą stowarzyszenia, do którego należą (jeśli przypisani).

```
CREATE VIEW Widok_KibiceStowarzyszenia AS
```

```
SELECT
```

```
    k.IdKibice,
```

```
    k.Imie,
```

```
k.Nazwisko,  
s.Nazwa AS NazwaStowarzyszenia  
FROM Kibice k  
LEFT JOIN Stowarzyszenia s ON k.NrStowarzyszenia =  
s.IdStowarzyszenia;  
GO
```

-- Pokazuje informacje o uczestnictwie kibiców w wydarzeniach (z biletem).

```
CREATE VIEW Widok_KibicZawody AS  
SELECT  
    kz.IdKibicZawody,  
    kz.NrZawodow,  
    k.Imie,  
    k.Nazwisko,  
    kz.CenaBiletu  
FROM KibicZawody kz  
JOIN Kibice k ON kz.Kibic = k.IdKibice;  
GO
```

-- Łączy dane o zakupach pamiątek przez kibiców oraz informacje o wydarzeniach, podczas których dokonano zakupu.

```
CREATE VIEW Widok_ZakupyPamiatki AS  
SELECT  
    zkp.IdZakupy,
```

```
k.Imie AS KlientImie,  
k.Nazwisko AS KlientNazwisko,  
p.Nazwa AS NazwaPamiatki,  
p.Cena,  
w.Nazwa AS NazwaZawodow  
FROM Zakupy zkp  
JOIN Kibice k ON zkp.NrKlienta = k.IdKibice  
JOIN Pamiatki p ON zkp.NrPamiatki = p.IdPamiatki  
JOIN Zawody w ON zkp.IdZawodow = w.IdZawody;  
GO
```

-- Prezentuje informacje o sponsoringu – łącząc dane o sponsorach oraz wydarzeniach.

```
CREATE VIEW Widok_Sponsoring AS  
SELECT  
    ssp.IdSponsoring,  
    sp.Nazwa AS Sponsor,  
    w.Nazwa AS NazwaZawodow,  
    ssp.Kwota  
FROM Sponsoring ssp  
JOIN Sponsorzy sp ON ssp.IdSponsora = sp.IdSponsorzy  
JOIN Zawody w ON ssp.Zawody = w.IdZawody;  
GO
```

-- Łączy informacje o konkursach dla kibiców z danymi o zwycięzcach oraz nagrodach.

CREATE VIEW Widok_KonkursyKibicow AS

SELECT

kd.IdKonkursyDlaKibicow,

kd.Nazwa AS NazwaKonkursu,

k.Imie AS ZwyciezcaImie,

k.Nazwisko AS ZwyciezcaNazwisko,

p.Nazwa AS Nagroda

FROM KonkursyDlaKibicow kd

JOIN Kibice k ON kd.Zwyciezca = k.IdKibice

JOIN Pamiatki p ON kd.NrNagrody = p.IdPamiatki;

GO

FUNKCJE:

CREATE FUNCTION dbo.F_RankingZawodnika(@IdZawodnika INT)

RETURNS INT

AS

BEGIN

DECLARE @Czas DECIMAL(4,2);

DECLARE @Ranking INT;

SELECT @Czas = Czas_s

FROM Rekord100m

WHERE NrZawodnika = @IdZawodnika;

IF @Czas IS NULL

RETURN NULL;

SELECT @Ranking = COUNT(*) + 1

FROM Rekord100m

WHERE Czas_s < @Czas;

RETURN @Ranking;

END;

GO

CREATE FUNCTION

dbo.F_SpodsumowanieWydatkowKibica(@IdKibice INT)

RETURNS TABLE

AS

RETURN

(

SELECT

@IdKibice AS IdKibice,

ISNULL((SELECT SUM(CenaBiletu)

FROM KibicZawody

WHERE Kibic = @IdKibice), 0) AS TotalTicketCost,

ISNULL((SELECT SUM(p.Cena)

```

FROM Zakupy z
JOIN Pamiatki p ON z.NrPamiatki = p.IdPamiatki
WHERE z.NrKlienta = @IdKibice), 0) AS TotalSouvenirCost,
ISNULL((SELECT SUM(CenaBiletu)
FROM KibicZawody
WHERE Kibic = @IdKibice), 0)
+ ISNULL((SELECT SUM(p.Cena)
FROM Zakupy z
JOIN Pamiatki p ON z.NrPamiatki = p.IdPamiatki
WHERE z.NrKlienta = @IdKibice), 0) AS OverallCost
);
GO

```

```

CREATE FUNCTION dbo.F_DyscyplinaPerformance(@IdDyscypliny INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        COUNT(*) AS LiczbaRekordow,
        MIN(r.Czas_s) AS NajlepszyCzas,
        MAX(r.Czas_s) AS NajgorszyCzas,
        AVG(r.Czas_s) AS SredniCzas
    FROM Rekord100m r

```

```
JOIN Zawodnicy z ON r.NrZawodnika = z.IdZawodnicy
WHERE z.GlownaDyscyplina = @IdDyscypliny
);
GO
```

--Procedury Skladowane

```
CREATE PROCEDURE DodajOsobeSportowa
```

```
@IdOsobaSportowa INT,
@Imie NVARCHAR(50),
@Nazwisko NVARCHAR(50),
@DataUrodzenia DATETIME,
@Status1 NVARCHAR(50)
```

```
AS
```

```
BEGIN
```

```
-- Sprawdź, czy data urodzenia jest w przeszłości
```

```
IF @DataUrodzenia >= GETDATE()
```

```
BEGIN
```

```
    RAISERROR('Data urodzenia musi być w przeszłości.', 16, 1);
```

```
    RETURN;
```

```
END;
```

```
-- Dodaj nową osobę sportową
```

```
INSERT INTO OsobaSportowa (IdOsobaSportowa, Imie, Nazwisko,
DataUrodzenia, Status1)
```



```
VALUES (@IdOsobaSportowa, @Imie, @Nazwisko,  
@DataUrodzenia, @Status1);
```

```
END;
```

```
GO
```

```
CREATE PROCEDURE UsunOsobeSportowa
```

```
    @IdOsobaSportowa INT
```

```
AS
```

```
BEGIN
```

```
    -- Sprawdź, czy osoba sportowa istnieje
```

```
    IF NOT EXISTS (SELECT 1 FROM OsobaSportowa WHERE  
IdOsobaSportowa = @IdOsobaSportowa)
```

```
    BEGIN
```

```
        RAISERROR('Osoba sportowa o podanym ID nie istnieje.', 16, 1);
```

```
        RETURN;
```

```
    END;
```

```
    -- Usuń osobę sportową (kaskadowo usunie zawodników i kibiców)
```

```
    DELETE FROM OsobaSportowa
```

```
    WHERE IdOsobaSportowa = @IdOsobaSportowa;
```

```
END;
```

```
GO
```

```
CREATE PROCEDURE AktualizujStatusOsobySportowej
```

```
@IdOsobaSportowa INT,  
@NowyStatus NVARCHAR(50)  
  
AS  
  
BEGIN  
    -- Sprawdź, czy osoba sportowa istnieje  
    IF NOT EXISTS (SELECT 1 FROM OsobaSportowa WHERE  
IdOsobaSportowa = @IdOsobaSportowa)  
    BEGIN  
        RAISERROR('Osoba sportowa o podanym ID nie istnieje.', 16, 1);  
        RETURN;  
    END;  
  
    -- Zaktualizuj status  
    UPDATE OsobaSportowa  
    SET Status1 = @NowyStatus  
    WHERE IdOsobaSportowa = @IdOsobaSportowa;  
END;  
  
GO
```

```
CREATE PROCEDURE PobierzZawodnikowZDyscypliny  
    @IdDyscyplina INT  
AS  
BEGIN
```

-- Sprawdź, czy dyscyplina istnieje

IF NOT EXISTS (SELECT 1 FROM Dyscyplina WHERE IdDyscyplina =
@IdDyscyplina)

BEGIN

RAISERROR('Dyscyplina o podanym ID nie istnieje.', 16, 1);

RETURN;

END;

-- Pobierz zawodników z danej dyscypliny

SELECT Z.IdZawodnicy, O.Imie, O.Nazwisko, D.Nazwa AS Dyscyplina
FROM Zawodnicy Z

INNER JOIN OsobaSportowa O ON Z.OsobaSportowaId =
O.IdOsobaSportowa

INNER JOIN Dyscyplina D ON Z.GlownaDyscyplina = D.IdDyscyplina
WHERE Z.GlownaDyscyplina = @IdDyscyplina;

END;

GO

CREATE PROCEDURE DodajZawody

@IdZawody INT,

@Nazwa VARCHAR(46),

@Data DATETIME,

@IdMiejsce INT

AS

```
BEGIN
```

```
-- Sprawdź, czy data zawodów jest w przeszłości
```

```
IF @Data >= GETDATE()
```

```
BEGIN
```

```
    RAISERROR('Data zawodów musi być w przeszłości.', 16, 1);
```

```
    RETURN;
```

```
END;
```

```
-- Sprawdź, czy miejsce zawodów istnieje
```

```
IF NOT EXISTS (SELECT 1 FROM Miejsce WHERE IdMiejsce =  
@IdMiejsce)
```

```
BEGIN
```

```
    RAISERROR('Miejsce o podanym ID nie istnieje.', 16, 1);
```

```
    RETURN;
```

```
END;
```

```
-- Dodaj nowe zawody
```

```
INSERT INTO Zawody (IdZawody, Nazwa, Data, Miejsce)
```

```
VALUES (@IdZawody, @Nazwa, @Data, @IdMiejsce);
```

```
END;
```

```
GO
```

```
SELECT Z.IdZawodnicy, O.Imie, O.Nazwisko, D.Nazwa AS Dyscyplina
```

```
FROM Zawodnicy Z
```

```
INNER JOIN OsobaSportowa O ON Z.OsobaSportowaId =  
O.IdOsobaSportowa
```

```
INNER JOIN Dyscyplina D ON Z.GlownaDyscyplina = D.IdDyscyplina;
```

```
SELECT Z.Nazwa AS Zawody, M.Nazwa AS Miejsce, Z.Data
```

```
FROM Zawody Z
```

```
INNER JOIN Miejsce M ON Z.Miejsce = M.IdMiejsce;
```

```
SELECT T.ZdobyteTrofeum, O.Imie, O.Nazwisko, D.Nazwa AS  
Dyscyplina
```

```
FROM Trofea T
```

```
INNER JOIN Zawodnicy Z ON T.NrZawodnika = Z.IdZawodnicy
```

```
INNER JOIN OsobaSportowa O ON Z.OsobaSportowaId =  
O.IdOsobaSportowa
```

```
INNER JOIN Dyscyplina D ON T.NrDyscyplinySportowej =  
D.IdDyscyplina;
```

```
-- zawodnicy u18
```

```
SELECT O.Imie, O.Nazwisko, O.DataUrodzenia
```

```
FROM OsobaSportowa O
```

```
INNER JOIN Zawodnicy Z ON O.IdOsobaSportowa =  
Z.OsobaSportowaId
```

```
WHERE YEAR(O.DataUrodzenia) > 2007;
```

```
SELECT D.Nazwa AS Dyscyplina, COUNT(Z.IdZawodnicy) AS  
LiczbaZawodnikow
```

```
FROM Zawodnicy Z
INNER JOIN Dyscyplina D ON Z.GlownaDyscyplina = D.IdDyscyplina
GROUP BY D.Nazwa;
```

WYZWALACZE:

```
1. CREATE TRIGGER TR_ustaw_na_osobe_akt
ON Zawodnicy
AFTER INSERT
AS
BEGIN
    UPDATE OsobaSportowa
    SET Status1 = 'Aktywny'
    FROM OsobaSportowa
    INNER JOIN inserted i ON OsobaSportowa.IdOsobaSportowa =
i.OsobaSportowald;
END;
GO
```

```
2. CREATE TRIGGER TR_zatwierdzenie_peselu
ON Zawodnicy
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (
```

```
SELECT 1
FROM inserted
WHERE PESEL LIKE '%[^0-9]%' OR LEN(PESEL) <> 11
)
BEGIN
    RAISERROR('Błędny format numeru PESEL. PESEL powinien
zawierać dokładnie 11 cyfr.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;
END;
GO
```

```
3. CREATE TRIGGER TR_zapobieganie_usunieciu_akt_zawodnika
ON OsobaSportowa
INSTEAD OF DELETE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM deleted d
        JOIN Zawodnicy z ON d.IdOsobaSportowa = z.OsobaSportowaId
        JOIN ZawodnicyZawody zz ON z.IdZawodnicy = zz.NrZawodnika
    )
```

```
BEGIN

    RAISERROR('Nie można usunąć osoby sportowej, która brała
udział w zawodach.', 16, 1);

    ROLLBACK TRANSACTION;

    RETURN;

END

ELSE

BEGIN

    DELETE FROM OsobaSportowa

    WHERE IdOsobaSportowa IN (SELECT IdOsobaSportowa FROM
deleted);

    END;

END;

GO
```

```
4. CREATE TRIGGER TR_Sprawdz_dostepnosc_miejsc

ON Zawody

AFTER INSERT

AS

BEGIN

    IF EXISTS (

        SELECT 1

        FROM Zawody z

        JOIN inserted i ON z.Miejsce = i.Miejsce
```



```
WHERE CAST(z.Data AS DATE) = CAST(i.Data AS DATE)
AND z.IdZawody <> i.IdZawody
)
BEGIN
    RAISERROR('W tym miejscu jest już zaplanowane wydarzenie w
    tym dniu.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END;
END;
GO
```

```
5. CREATE TRIGGER TR_nowy_rekord_100m
ON Rekord100m
AFTER INSERT
AS
BEGIN
    DECLARE @NewTime DECIMAL(4,2);
    DECLARE @BestTime DECIMAL(4,2);

    SELECT TOP 1 @NewTime = Czas_s FROM inserted;
    SELECT @BestTime = MIN(Czas_s) FROM Rekord100m;

    IF @NewTime = @BestTime
```

```
BEGIN
    PRINT 'Nowy rekord w biegu na 100m! Gratulacje!';
END;
END;
GO
```

INDEKSY:

```
CREATE INDEX idx_osoba_sportowa ON OsobaSportowa
(IdOsobaSportowa);
CREATE INDEX idx_glowna_dyscyplina ON Zawodnicy
(GlownaDyscyplina);
CREATE INDEX idx_nr_zawodnika ON ZawodnicyZawody
(NrZawodnika);
CREATE INDEX idx_id_zawodow ON Zawody (IdZawody);
CREATE INDEX idx_kibic ON Kibice (OsobaSportowaId);
CREATE INDEX idx_pamiatki ON Pamiatki (TypPamiatki);
```